

Machine learning

Lecture 2 - Data Imputation

03.04.2025

Toker Gilat



Data Imputation

Outline

- **Why Missing Data is a Problem**
- **What is missing data**
- **Why do we have missing data**
- **How to detect missing data**
- **Types of missing data**
- **What is imputation**
- **Techniques of imputation**

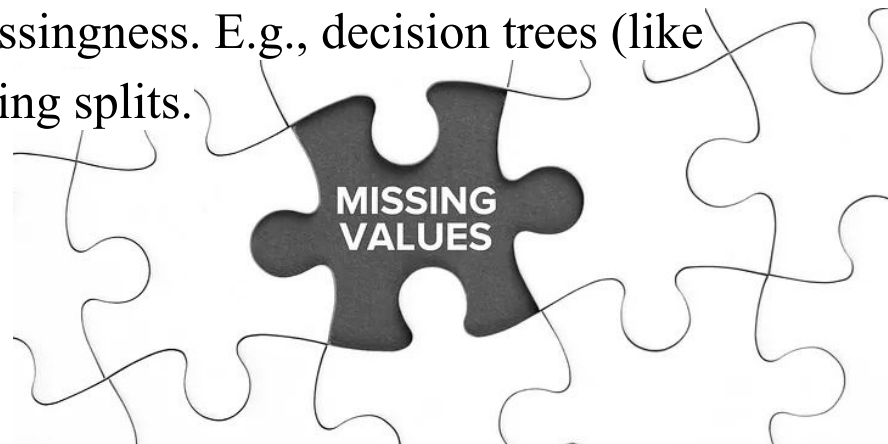
Why Missing Data is a Problem

Main Points:

- Loss of Information
- Bias in Results - If the data is not missing completely at random
- Impact on Model Training - Algorithms expect complete input, Skews distributions, Removes patterns

What is missing data

- Missing data occurs when no value is stored for one or more features in a dataset.
- Models that use this dataset might become **inaccurate**
- Any data that is lost in the database can obscure the discovery of important factors, bias the data and lead to invalid conclusions and models
- Some models can directly model the missingness. E.g., decision trees (like XGBoost) can internally deal with missing splits.



Why do we have missing data

- **Nonresponse in Surveys** - Respondents may accidentally or intentionally skip questions, provide incomplete answers, or fail to return the questionnaire.
- **Dropout or Noncompliance in Clinical Trials** - Participants may drop out, miss visits, or stop treatment. (Possible Reasons: Side effects, lack of perceived effectiveness, or personal circumstances).
- **Data Labeling Issues** - Not all data points are labeled, often due to resource constraints.
- **Human Factors** - Sensitive or disliked topics can lead to skipped responses, skewing results.
- **Errors**: Human Errors: Data entry mistakes, incomplete surveys. System Errors: Faulty sensors, data corruption.
- **Sensitive Information**: Certain fields may be left blank intentionally, especially for privacy reasons.

How to detect missing data

Common Representations of Missing Data

1. **Explicit Missing Values:** 'NaN' (Not a Number), 'Null'
2. **Implicit Missing Values:** '-999', '0', '1', '?', 'N/A', 'inf', '-inf'
3. **Incomplete Records:** Rows or columns with fewer entries than expected.

Many libraries are available for your use (e.g. missingno (correlation between missingness of different columns))

Types of missing data

Scenario: We conduct a survey to understand the satisfaction level of employees in a company. The survey includes questions about their job satisfaction, workload, and overall well-being.

Observation: We notice that some employees did not respond to the survey, and you want to understand the nature of the missing data

Unobserved Data: Employees' true job satisfaction levels, especially those who are very dissatisfied.

Observed Data: The responses from employees who chose to complete the survey.

Missing data mechanism describes the underlying mechanism that generates missing data and can be categorized into three types.

Types of missing data

1. Missing Completely at Random (MCAR)

Definition: The likelihood of data being missing is entirely independent of both observed and unobserved data.

Example: Some employees didn't fill out the survey because they were randomly selected for a training program on the survey day. The missing data is not related to their job satisfaction or any other variables in the survey.

Types of missing data

2. Missing at Random (MAR)

Definition: The likelihood of data being missing is related to observed data but not to the missing data itself.

Example: Employees with higher workloads (an observed variable) are less likely to complete the survey because they are too busy. The missing data is related to workload but not directly to their job satisfaction.

Types of missing data

3. Missing Not at Random (MNAR)

Definition: The likelihood of data being missing is related to the unobserved data.

Example: Employees who are very dissatisfied with their job (an unobserved variable) are less likely to complete the survey because they fear retaliation or believe it won't make a difference. The missing data is related to their job satisfaction, which is not observed in those who did not respond.

Main problem:

There is no way that we can distinguish whether the missing data is MCAR or MNAR from the observed data.

Types of missing data

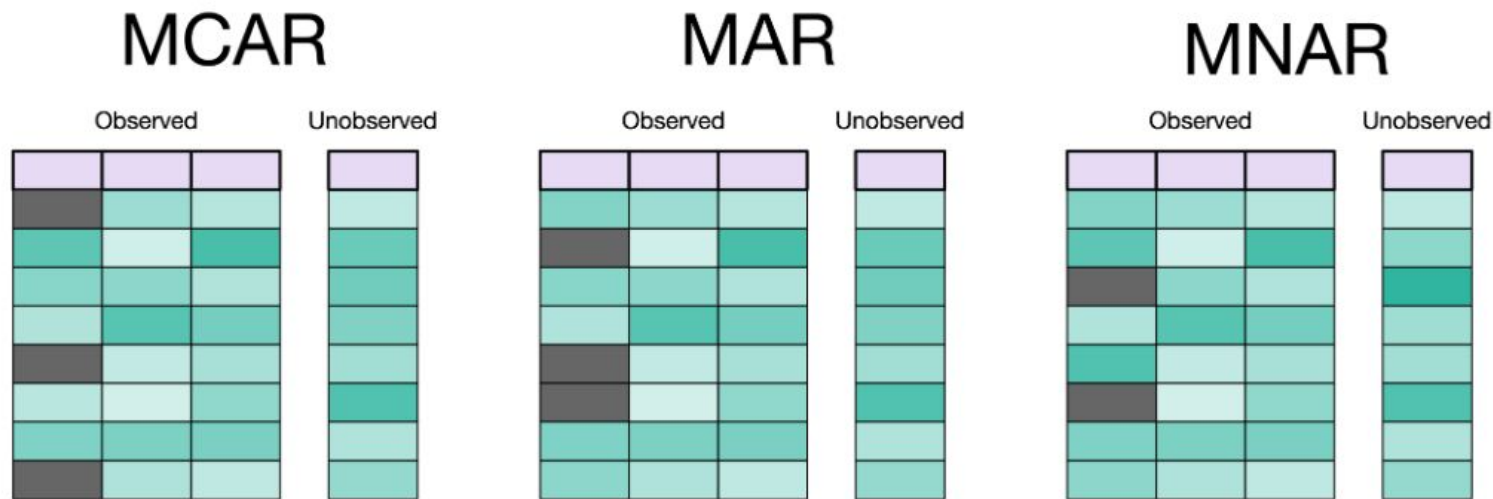


Figure 42.1: The 3 types of missing data MCAR, MAR, and MNAR. Missing data is represented in the first column as gray squares. The missing values don't seem to have a pattern for MCAR. For MAR the missing values appear to align with light values in the second column. For MNAR we have that the missing values align with dark values of the unobserved data.

What is imputation

One of the most common ways of dealing with missing values is to fill them in with some values. The idea is :

1. To impute (or estimate) missing values based on observed data
2. Then to carry out the analysis that would be undertaken if the full data were all observed

What Makes a “Good” Imputation?

- Plausibility: Values should make sense (e.g., age shouldn't be -10).
- Preservation of distribution: Shouldn't distort means/variances too much.
- Low bias introduction: Especially important if MAR/MNAR.

Techniques of imputation

1. Complete Case Analysis

Ignore the missing values. Only use complete data samples

	A	B	C	D
1	Gender	Price	Make	Engine
2	Female	100	Ford	2000
3	Male		Ford	
4	Male	50	Kia	1500
5	Female	60		1500
6	Female	120	Nissan	3000
7	Male	200	BMW	4500



	A	B	C	D
1	Gender	Price	Make	Engine
2	Female	100	Ford	2000
3	Male	50	Kia	1500
4	Female	120	Nissan	3000
5	Male	200	BMW	4500

Techniques of imputation

2. Dummy Variable Adjustments

- Plug in some arbitrary value for all missing values: 0
- Mean of the values of the variable that are not missing
- Median
- Mode (most frequently appeared value) -> for discrete variables

Optional: Include in the model a dummy variable coded 1 if data in the original variable was missing, 0 otherwise

Techniques of imputation

3. Hot-Deck Imputation

Replace missing values by values extracted from cases that are similar to it.

- K-Nearest-Neighbors
 - A KNN model is trained using complete data.
 - Complete missing values with the most frequent value/mean among the k nearest neighbors.
- LOCF (last observation carried forward)

Good for longitude data (E.g., a patients' weights over a period of visits)

Techniques of imputation

4. Regression Imputation

If we know there is a correlation between the missing value and other variables, we can often get better guesses by regressing the missing variable on other variables.

1. Calculate correlation between the missing variable and other variables
2. Choose the correlated variables and build a regression model using them as features, and the missing variable as target
3. Predict the missing values using the model

Techniques of imputation

There are other more advanced methods that combine the ideas of the basic methods that we have discussed above.

Problems related to missing data are opportunities for researchers to get the right techniques to overcome them.

Method	Approach	Computing Cost	Attributes
Case Deletion	-	Low	Num & Nom
Maximisation Likelihood Methods	Statistic	Low	Num
Mean Imputation	Statistic	Low	Num & Nom
All Possible Values Imputation	Statistic	High	Nom
Regression Methods	Statistic	Low	Num
Hot deck imputation	ML	Low	Num & Nom
K-Nearest Neighbor Imputation	ML	High	Num
Multiple Imputation	ML	High	Num & Nom
Internal treatment method for C4.5	ML	Midle	Num & Nom
Bayesian Iteration Imputation	ML	Midle	Nom

Pandas Recap

Pandas Recap

1. **Data Selection and Filtering**
2. **Data Transformation**
3. **Aggregation and Grouping**
4. **Combining Data Frames**
5. **Handling Missing Data**
6. **Input and Output Operations**

Pandas Recap

Data Selection and Filtering

.loc[]: Select by label.

.iloc[]: Select by position.

```
df.loc[0, 'column_name']  
df.iloc[0, 0]
```

Pandas Recap

Data Transformation

- .apply():** Apply a function along an axis.
- .map():** Element-wise transformations for a Series.
- .replace():** Replace values.
- .fillna():** Fill missing values.

```
df['column'].apply(lambda x: x + 1)
df['column'] = df['column'].map({'A': 'Apple', 'B': 'Banana'})
df['column'].fillna(0)
```

Pandas Recap

Aggregation and Grouping

.groupby(): Group data for aggregation.

.agg(): Aggregate using one or more operations.

.sum(), .mean(), .min(), .max(): Common aggregation functions.

```
df.groupby('column').sum()
```

```
df.groupby('column').agg({'A': 'sum', 'B': 'mean'})
```

Pandas Recap

Combining Data Frames

pd.concat(): Concatenate pandas objects.

pd.merge(): Merge DataFrames.

.join(): Join columns of another DataFrame.

```
pd.concat([df1, df2])  
pd.merge(df1, df2, on='key')  
df1.join(df2, on='key')
```


Pandas Recap

Handling Missing Data

.dropna(): Remove missing values.

.fillna(): Fill missing values.

```
df.dropna()  
df.fillna(0)
```

Pandas Recap

Input and Output Operations

pd.read_csv(): Read a CSV file.

.to_csv(): Write to a CSV file.

```
df = pd.read_csv('data.csv')  
df.to_csv('output.csv')
```

Hands on !



Open Google colab notebook -
Exploratory Data Analysis
ML_L2_Imputation.ipynb

Scikit-Learn

Scikit-Learn

Datasets

Scikit-learn provides access to several standard datasets like Iris, digits, and Boston housing.

```
# Import necessary libraries
from sklearn.datasets import load_iris
import pandas as pd

# Load the Iris dataset
iris = load_iris()

# Convert the dataset to a Pandas DataFrame
iris_df = pd.DataFrame(data=iris.data, columns=iris.feature_names)

# Add the target variable to the DataFrame
iris_df['species'] = iris.target

# Display the first few rows of the DataFrame
print(iris_df.head())
```

Scikit-Learn

preprocessing transformers

Transformers in scikit-learn are used for data preprocessing, implementing both *fit* and *transform* methods to prepare data for modeling.

- **StandardScaler:** Standardizes features by removing the mean and scaling to unit variance.
- **MinMaxScaler:** Scales features to a specified range.
- **PCA** (Principal Component Analysis): Reduces dimensionality.
- **OneHotEncoder:** Encodes categorical features as a one-hot numeric array.

Scikit-Learn

preprocessing transformers

```
from sklearn.preprocessing import StandardScaler
import numpy as np

# Sample data
data = np.array([[1, 2], [3, 4], [5, 6]])

# Initialize the StandardScaler
scaler = StandardScaler()

# Fit the scaler to the data
scaler.fit(data)

# Transform the data
scaled_data = scaler.transform(data)

print(scaled_data)
```

What will be the output?

Scikit-Learn

Estimators

Estimators are used for modeling and prediction tasks.

They provide implementations for regression, classification, and clustering models.

Core Methods of Estimators:

- ***fit***: Trains the model on the provided data.
- ***predict***: Generates predictions based on the trained model.
- ***score***: Evaluates model performance on test data.

Common Scikit-Learn Estimators:

- ***LinearRegression***: For regression tasks, predicting continuous values.
- ***LogisticRegression***: For binary and multi-class classification.
- ***SVM***: Used for classification and regression with margin-based learning.
- ***RandomForestClassifier***: An ensemble model for classification tasks.

Scikit-Learn

Estimators

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data
X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
y = np.dot(X, np.array([1, 2])) + 3

# Initialize the LinearRegression model
model = LinearRegression()

# Fit the model to the data
model.fit(X, y)

# Make predictions
predictions = model.predict(X)

print(predictions)
```

Scikit-Learn

Estimators

```
from sklearn.linear_model import LinearRegression
import numpy as np

# Sample data
X = np.array([[1, 1], [1, 2], [2, 2], [2, 3]])
y = np.dot(X, np.array([1, 2])) + 3

# Initialize the LinearRegression model
model = LinearRegression()

# Fit the model to the data
model.fit(X, y)

# Make predictions
predictions = model.predict(X)

print(predictions)
```

More about
sklearn later on.